

PATENT RETRIEVER

Technical Documentation

A Python application that retrieves patent documents from multiple sources, normalizes them into a single internal model, and generates a precisely formatted Word document for a patent translation pipeline, alongside a machine-readable JSON representation.

Built with Python 3.13, Flask, and python-docx

Sources: Google Patents · EPO Open Patent Services

Contents

Contents	2
1. Overview.....	4
1.1 Capability summary	4
2. Solution Architecture.....	6
2.1 Design principles.....	6
2.2 Component responsibilities.....	7
2.3 The retrieval pipeline.....	7
3. Technology Stack	9
3.1 Repository structure	10
4. The Domain Model	11
4.1 PatentDocument	11
4.2 Section mapping	11
5. Data Sources.....	13
5.1 The source contract	13
5.2 Google Patents	13
5.3 EPO Open Patent Services	14
5.4 Adding a source	15
6. The Output Format.....	16
6.1 Measured specification	16
6.2 Two-stage rendering.....	16
6.3 Four-digit list numbering	16
6.4 JSON output.....	17
7. Interfaces.....	18
7.1 Command-line interface	18
7.2 Web interface	18
8. The Generated Document	20
8.1 A data defect worth documenting	22
9. Design Decisions	24
9.1 A frozen dataclass rather than Pydantic.....	24
9.2 Completeness is a service concern	24
9.3 Presentation lives in the renderer	24
9.4 Returning bytes rather than writing files.....	24
9.5 Non-English full text is used, with a warning.....	24
9.6 The country limitation is discovered, not hard-coded.....	24
9.7 Claim boundaries by prefix, not by structure	25
9.8 Selective markup stripping	25
10. Error Handling	26

10.1 Mapping external failures.....	26
10.2 Exit codes.....	26
11. Testing Strategy.....	27
11.1 Principles	27
11.2 Coverage by area	27
11.3 Deliberately not tested.....	27
12. Results and Validation	29
12.1 Automated checks	29
12.2 End-to-end verification.....	29
12.3 Document validity	29
12.4 Continuous integration.....	30
13. Deployment and Setup	31
13.1 Prerequisites.....	31
13.2 Installation	31
13.3 Configuration.....	31
13.4 Usage	31
13.5 Development	31
14. Limitations	32
14.1 Sources	32
14.2 Output	32
14.3 Deployment.....	32
15. Future Work	32

1. Overview

Preparing a patent for translation is a transcription task. Somebody opens a patent database, copies the title, the abstract, the claims, and the description into a Word document, and reformats the result to match a template that a downstream translation tool expects. The work is mechanical, it takes a long time for a document that routinely runs to two hundred paragraphs, and the mistakes it produces — a dropped claim, a renumbered paragraph, a heading in the wrong case — are invisible until the translation comes back wrong.

Patent Retriever removes that step. Given a publication number, it retrieves the patent from an external source, converts it into a single internal representation, and writes two files: a Word document that matches the required template exactly, and a JSON file carrying the same content in a form other tools can consume.

The application does not translate anything. It prepares the input, and that boundary is deliberate — translation is a separate concern owned by a separate system.

Two sources are supported and are tried in a configured order. Google Patents is scraped; EPO Open Patent Services is queried through its authenticated REST API. Adding a third requires implementing a single method and registering it; no other part of the application changes.

1.1 Capability summary

Capability	Description
Multi-source retrieval	Tries each configured source in order until one returns a document that passes the completeness policy.
Input normalization	Accepts a publication number in any common form and reduces it to a canonical value before any request is made.
Unified domain model	Every source produces the same immutable PatentDocument, so nothing downstream knows where the data came from.
Section mapping	Maps the many heading conventions used by real patents onto the four sections the output template requires.
Template-exact Word output	Reproduces the target document precisely: page setup, typography, a running page header, and genuine Word list numbering.
JSON handoff	Emits the same content as structured JSON with a schema version, for consumption by other tools.
Command-line interface	A single command produces both files, with distinct exit codes for user error, retrieval failure, and misconfiguration.
Web interface	A browser form for retrieval, with a document preview and direct download of either format.
Graded failure handling	An exception hierarchy distinguishes failures that should advance the fallback chain from those that should stop immediately.
Configuration by environment	Source order, timeouts, credentials, and output location are read from the environment; behaviour changes require no code change.

Design intent

The application is built so that the volatile parts stay at the edges. HTML markup changes, APIs are versioned, document templates get revised — none of that should reach the centre of the codebase.

The result is a core that has no dependencies outside the Python standard library, and adapters around it that can be replaced individually. Section 9 records the decisions that follow from this constraint.

2. Solution Architecture

The architecture is a layered design in which every dependency points inward. Interfaces depend on the service; the service depends on sources and renderers; all of them depend on the domain. The domain depends on nothing — it imports only the Python standard library.

That single rule is what makes the rest of the system tractable. Google Patents can change its markup, EPO can version its API, the Word template can be revised, and each of those changes is contained inside one module.

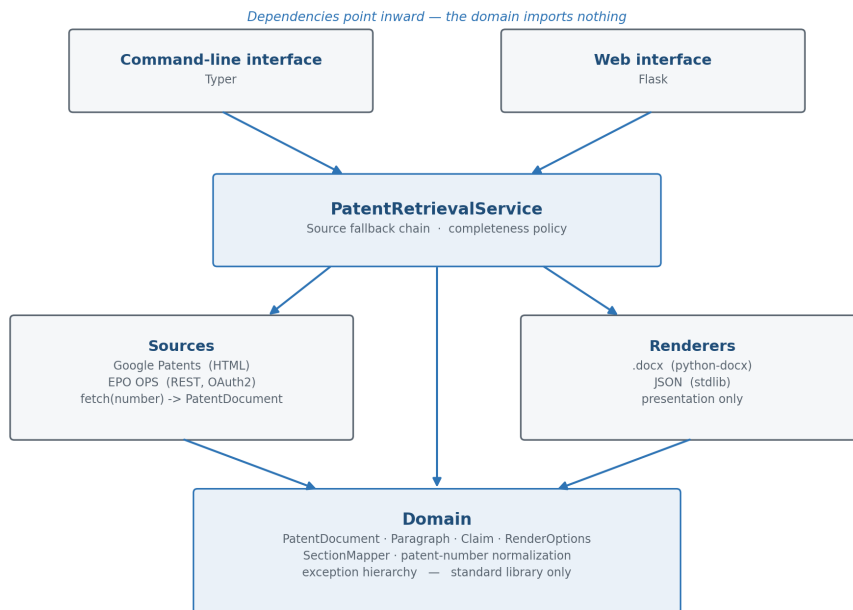


Figure 1. Layered architecture. Every arrow points inward toward a domain that has no outward dependencies.

2.1 Design principles

- Dependency inversion. The core defines what a patent is; adapters conform to it. No domain module imports `httpx`, `BeautifulSoup`, `python-docx`, or `Flask`.
- Single responsibility per module. A source fetches and parses. A renderer formats. The service orchestrates. None of them does two of those things.
- Immutability at the centre. `PatentDocument` is a frozen dataclass holding tuples. Once a source returns it, nothing downstream can alter it.
- Validation at construction. An invalid document cannot be instantiated, so no consumer defends against a missing title or an empty claim list.
- Policy in one place. What counts as a complete document is decided by the service, not scattered across the sources.
- Configuration over code. Source order, credentials, timeouts, and paths come from the environment.
- Fail forward. A source failure advances the chain rather than ending the run.

2.2 Component responsibilities

Layer	Component	Responsibility	Must not
Domain	models	PatentDocument, Paragraph, Claim, RenderOptions and their validation.	Perform any I/O.
Domain	sections	Map arbitrary source headings onto the four canonical output sections.	Know which source produced them.
Domain	patent_number	Normalize and validate publication numbers.	Contact a network.
Domain	exceptions	The exception hierarchy that drives fallback behaviour.	Reference any adapter.
Sources	base	The PatentSource protocol: fetch(number) -> PatentDocument.	Contain an implementation.
Sources	google_patents	Retrieve and parse a Google Patents page.	Know other sources or output formats.
Sources	epo_ops	Authenticate and query EPO OPS, then parse its responses.	Know other sources or output formats.
Sources	registry	Build source instances from their configured names.	Decide retrieval order at runtime.
Services	retrieval	Run the fallback chain; apply the completeness policy.	Parse markup or write files.
Renderers	docx_renderer	Turn a PatentDocument into template-exact Word bytes.	Fetch anything or mutate the document.
Renderers	json_renderer	Turn a PatentDocument into a versioned JSON envelope.	Apply document formatting.
Interfaces	cli	Parse arguments, call the service, write files, report.	Contain business logic.
Interfaces	web	Serve the form, preview, and downloads.	Contain business logic.

2.3 The retrieval pipeline

1. The supplied publication number is normalized to a canonical form. A number that cannot be one is rejected here, before any network request is made.
2. The service takes the first configured source and calls its fetch method with the normalized number.
3. The source performs whatever it must — an HTTP GET and an HTML parse, or an OAuth2 token exchange followed by three REST calls — and returns a PatentDocument, or raises.
4. Anything raised beneath SourceError advances the chain. The failure is logged with its reason and the next source is tried.
5. A returned document is checked against the completeness policy: abstract, detailed description, and inventors must be present.
6. A complete document is returned immediately. An incomplete one is retained as a candidate while the remaining sources are tried.

7. If no source produces a complete document, the best candidate is returned with a record of what is missing. Only if every source failed outright is an error raised.

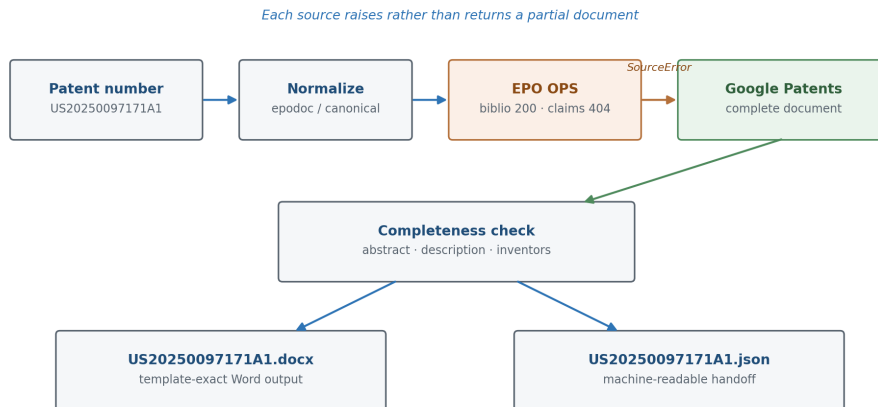


Figure 2. The fallback chain in practice. EPO OPS serves bibliographic data for a United States publication but refuses its full text, so the chain advances to Google Patents.

Returning the best incomplete candidate rather than failing is a deliberate choice. A partial document that names its own gaps is more useful to somebody preparing a translation than an error message, and the caller is always told which fields are missing.

3. Technology Stack

Every dependency in the list below earns its place by doing something the standard library does not. The core of the application uses none of them.

Component	Role	Why this one
Python 3.13	Runtime	Modern typing syntax, StrEnum, and dataclass slots without third-party support.
httpx	HTTP client	Sane timeout semantics and a requests-compatible API, with async available if it is ever needed.
BeautifulSoup 4 + lxml	HTML parsing	Tolerant of the imperfect markup that scraped pages always contain.
python-docx	Word generation	The only practical library for programmatic .docx creation.
Flask	Web interface	Proportionate to a single-form internal tool; a heavier framework would not earn its cost.
Typer	Command-line interface	Type-hint driven, so the CLI signature is the argument specification.
python-dotenv	Configuration	Keeps credentials in an ignored .env file rather than in code.
pytest + respx	Testing	respx intercepts httpx traffic, so source tests run against saved fixtures with no network.
ruff	Linting and formatting	Replaces flake8, isort, and black with one fast tool.
mypy (strict)	Static type checking	Catches the None-handling errors that patent data is full of.
GitHub Actions	Continuous integration	Runs the full check suite on a clean checkout after every push.

Deliberately not used: Pydantic, because validation here is a handful of lines and a third-party import in the domain would reverse the dependency arrows; Scrapy, because the application fetches one page rather than crawling; Selenium or Playwright, because Google Patents renders its content server-side; and any database, because there is nothing to persist.

3.1 Repository structure

```

$ (venv) Abdulrahman-Madoun-Air-2:patent-retriever zuneis clear && find . -type d \! -name .venv -o -name .git -o -name '__pycache__' -o -name 'output' -o -name '*.cache' \! -prune -o -print | sort | head -60
./
./_BG_Store
./claude
./claude/settings.local.json
./env
./env.example
./github
./github/workflows
./github/workflows/ci.yml
./gitignore
./build-environment
./CLAUDE.md
./LICENSE
./README.md
./docs
./docs/images
./docs/images/form.png
./docs/images/result.png
./docs/output-format.md
./project-tool
./scripts
./scripts/check_epo.py
./scripts/inspect_fixture.py
./scripts/make_sample.py
./src
./src/_BG_Store
./src/patent_retriever
./src/patent_retriever/__init__.py
./src/patent_retriever/config.py
./src/patent_retriever/domain
./src/patent_retriever/domain/__init__.py
./src/patent_retriever/domain/exception.py
./src/patent_retriever/domain/models.py
./src/patent_retriever/domain/patent_number.py
./src/patent_retriever/domain/function.py
./src/patent_retriever/interfaces
./src/patent_retriever/interfaces/__init__.py
./src/patent_retriever/interfaces/cli.py
./src/patent_retriever/interfaces/web
./src/patent_retriever/interfaces/web/__init__.py
./src/patent_retriever/interfaces/web/app.py
./src/patent_retriever/interfaces/web/templates
./src/patent_retriever/interfaces/web/templates/base.html
./src/patent_retriever/interfaces/web/templates/index.html
./src/patent_retriever/interfaces/web/templates/result.html
./src/patent_retriever/renderers
./src/patent_retriever/renderers/__init__.py
./src/patent_retriever/renderers/doc_renderer.py
./src/patent_retriever/renderers/json_renderer.py
./src/patent_retriever/services
./src/patent_retriever/services/__init__.py
./src/patent_retriever/services/retrieval.py
./src/patent_retriever/sources
./src/patent_retriever/sources/__init__.py
./src/patent_retriever/sources/base.py
./src/patent_retriever/sources/epo.py
./src/patent_retriever/sources/google_patents.py
./src/patent_retriever/sources/registry.py
./tests
./tests/conftest.py

```

Figure 3. Repository layout. The *src/* layout forces the package to be installed rather than imported from the working directory, which surfaces packaging errors locally instead of in CI.

4. The Domain Model

The domain answers one question: what is a patent, as far as this application is concerned? Its answer is deliberately narrower than what any source offers. Google Patents exposes citations, legal status, and patent families; none of that is translated, so none of it appears in the model.

4.1 PatentDocument

```
@dataclass(frozen=True, slots=True)
class PatentDocument:
    publication_number: str
    title: str
    claims: tuple[Claim, ...]
    inventors: tuple[str, ...] = ()
    abstract: str | None = None
    abstract_figure_reference: str | None = None
    background: tuple[Paragraph, ...] = ()
    summary: tuple[Paragraph, ...] = ()
    brief_description_of_drawings: tuple[Paragraph, ...] = ()
    detailed_description: tuple[Paragraph, ...] = ()
```

Three properties of that declaration matter more than the field list.

Immutability is real, not nominal

The dataclass is frozen and every collection is a tuple. A frozen dataclass holding lists would still permit `document.claims.append(...)`, which is the kind of mutation that produces bugs nobody can trace. Tuples also make the empty default safe, avoiding the mutable-default trap entirely.

Transformations return new documents. The `renumbered` method, which reassigns paragraph numbers into the unbroken sequence the output template requires, produces a copy and leaves the original untouched.

Illegal states are unrepresentable

Validation runs in `__post_init__`. A publication number or title that is blank, an empty claim list, or paragraph numbers that do not strictly increase all raise at construction. The consequence is that every function downstream can assume a valid document, so the codebase contains no defensive checks for missing titles.

The same reasoning drives the optionality of each field. A missing abstract is `None`, never an empty string — because an empty string cannot distinguish between a patent that has no abstract and a scraper whose selector silently broke.

Presentation is not stored

Paragraph numbers are plain integers. The four-digit bracketed form the Word document requires is generated by the renderer, and would be noise in the JSON output. Similarly, the attorney docket number and the client reference are not fields on `PatentDocument`: they are properties of a rendering job, not of the patent, and live in a separate `RenderOptions` value.

4.2 Section mapping

The output template requires exactly four body sections. Real patents do not cooperate. They use TECHNICAL FIELD, FIELD OF THE INVENTION, DESCRIPTION OF RELATED ART, BRIEF SUMMARY, MODES FOR CARRYING OUT THE INVENTION, and many other conventions.

SectionMapper resolves this. It normalizes a heading, matches it against ordered keyword rules, and returns one of the four canonical sections or nothing. Content under an unrecognized heading continues the section currently being filled, so nothing is discarded and document order is preserved.

```
_RULES = (  
  (Section.DETAILED,    ("DETAILED", "EMBODIMENT", "BEST MODE", "MODES FOR CARRYING")),  
  (Section.DRAWINGS,    ("DRAWING", "FIGURE")),  
  (Section.SUMMARY,     ("SUMMARY", "DISCLOSURE OF THE INVENTION")),  
  (Section.BACKGROUND,  ("BACKGROUND", "FIELD", "RELATED ART", "PRIOR ART", ...)),  
)
```

Rule order is load-bearing. DETAILED DESCRIPTION OF THE DRAWINGS contains both DETAILED and DRAWING; evaluating the detailed-description rule first resolves it correctly. The module has no I/O at all, which is precisely why it can be tested exhaustively against real heading strings without a network.

5. Data Sources

5.1 The source contract

Every source satisfies one protocol. It is expressed as a typing Protocol rather than an abstract base class, so an implementation conforms by shape and inherits nothing — coupling stays at zero and test doubles are trivial.

```
@runtime_checkable
class PatentSource(Protocol):
    @property
    def name(self) -> str: ...

    def fetch(self, number: str) -> PatentDocument:
        """Retrieve a patent.

        Raises:
            PatentNotFoundError: the source has no record of this patent.
            SourceUnavailableError: the source could not be reached.
            ParseError: the response could not be interpreted.
        """
```

The contract is as notable for what it omits. A source is not asked whether its result is complete, does not know that other sources exist, and never learns what will be done with the document it returns.

5.2 Google Patents

Google Patents publishes no API, but its pages are rendered server-side, so an HTTP request and an HTML parse are sufficient — no browser automation is required. Parsing is a pure function of the HTML, separate from fetching, which is what allows it to be tested against a saved page with no network access.

Data	Selector	Note
Publication number	meta[name=citation_patent_publication_number]	Colon-delimited; separators are removed.
Title	meta[name=DC.title]	Whitespace-normalized.
Inventors	meta[name="DC.contributor"][scheme="inventor"]	The scheme attribute separates inventors from the assignee.
Abstract	div.abstract	A trailing figure reference is split off for the template.
Description	section[itemprop=description]	Walked in document order.
Section headings	heading	Fed to SectionMapper.
Paragraphs	div.description-line	The num attribute carries the original identifier.
Claims	div.claim[id]	The id filter excludes the wrapper element.

Claims are renumbered from one by position. Google prefixes claim text with its own number, which is stripped, because the output template requires an unbroken sequence that the source cannot be relied upon to supply.

This source is the fragile one. It depends on markup that can change without notice, and scraping sits uneasily with the site's terms of use. The architecture answers that directly: source order is configuration, so another source can be promoted ahead of it without touching code.

5.3 EPO Open Patent Services

EPO OPS is an official, authenticated REST API — structurally more reliable than scraping, and correspondingly more particular. Four behaviours had to be established empirically against live responses before the adapter could be written.

Reference normalization

OPS references in epodoc format omit the kind code, and United States publications additionally lose the leading zero of their seven-digit serial.

US20250097171A1	->	US2025097171	# publication: kind code and leading zero removed
EP0000001A1	->	EP0000001	# kind code removed
US11234567B2	->	US11234567	# grant: serial left intact

Getting this wrong is punished misleadingly. A malformed reference returns HTTP 404 with `SERVER.EntityNotFound` — "no results found" — which reads as a missing patent rather than a malformed request. Because the kind code is dropped from the request, it is read back off the original number and used to select the right publication from the response: EP0000001 returns both the A1 and the B1, with different titles and different full text.

Response shape

The JSON is a mechanical translation of XML, and every one of its quirks required handling.

Quirk	Handling
Text under "\$", attributes under "@name"	A recursive text extractor follows "\$" and skips any key beginning with "@".
Any element may be an object or a list	A wrapper coerces to a list. Both forms occur: EP returns three titles, US returns one.
Full text per language, German first	English is selected explicitly by language attribute, case-insensitively.
No English variant at all	The available language is used and a warning is logged naming it.
One reference, several publications	The publication is selected by kind code taken from the requested number.
claim-text entries are lines, not claims	Claims are split on a numbered line; other lines append to the claim above.
Parties repeated per data format	Inventors are taken from the epodoc format alone.
Bracketed paragraph markers in text	Stripped, because the renderer regenerates them as a Word list.
HTML entities and image placeholders	Entities are unescaped first, so escaped markers are then recognized and removed.

The claim-splitting rule deserves a note. OPS nests every line of every claim under a single element, so structure carries no information at all — the number prefix is the only boundary marker available. The rule requires a space after the period, so that a continuation line beginning "10.5 cm" cannot be mistaken for the start of claim ten. In one test document, 21 line entries carry 12 claims.

Inventor names cannot be deduplicated by string comparison either. OPS repeats each person once per data format with different spellings — BUSSE CLAUS-ADOLF, BUSSE CLAUS ADOLF, and BUSSE, CLAUS ADOLF, DR. are one inventor. Selecting the epodoc format yields exactly one entry per person.

The country limitation

EPO OPS does not serve United States full text

Bibliographic data and abstracts are returned for US publications. Claims and description return HTTP 404 with `CLIENT.InvalidCountryCode`. Full text is available for EP and WO documents only.

Because the domain model requires at least one claim, EPO can never produce a valid document for a US patent. It raises, and the chain advances to Google Patents — which is exactly the behaviour the fallback design exists to provide.

This limitation is discovered from the API rather than hard-coded. A country check in the source would go stale silently if OPS ever extended its coverage; the live response is the authority.

Unheaded descriptions

OPS text-only full text carries no heading markup, and the documents examined contain no heading paragraphs of any kind — not even in title case. There is nothing for `SectionMapper` to split on.

Left alone, the whole description falls to the mapper's default of `BACKGROUND`, which is correct for Google Patents — text before the first heading there genuinely is background material — but wrong here, where the entire body is the detailed description. The source therefore labels its own content: when no heading is recognized anywhere, it returns a single section headed `DETAILED DESCRIPTION`.

The alternative was to loosen the shared completeness policy to accept body content in any section. That was rejected on two grounds: it changes a policy common to every source in order to accommodate one source's quirk, and it leaves the generated document with everything filed under a heading the template does not intend.

5.4 Adding a source

Adding a third source touches two files. A new module in the `sources` package implementing `fetch`, and one entry in the registry that maps its configured name to a builder. The service, the renderers, the domain, and both interfaces are untouched — a claim the EPO OPS work verified in practice rather than in principle.

6. The Output Format

The Word document is the reason the application exists, and its specification was not available in written form. It was established by inspecting a real sample in Word — page setup, style definitions, paragraph properties, and the underlying numbering definitions — and recorded in the repository so the renderer can be tested against a written specification rather than a memory of one.

6.1 Measured specification

Property	Value
Page size and margins	A4 portrait; 1.2" top, 1" bottom, left and right; no gutter.
Body typography	Times New Roman 13 pt, 1.5 line spacing, 0 pt before, 6 pt after, justified.
Running page header	Right-aligned on every page: PATENT APPLICATION, ATTORNEY DOCKET NO., CLIENT REF. NO.
Cover block	Four empty paragraphs, then APPLICATION / FOR / UNITED STATES LETTERS PATENT centred at 18 pt bold.
Title and inventor labels	14 pt bold; TITLE: is followed by two tab characters before the value.
Title repetition	The title repeats, centred and bold, at the head of the body after a page break.
Section headings	Centred, 13 pt, not bold — except ABSTRACT, which is 14 pt bold.
Heading separation	No empty paragraph between a heading and its first paragraph.
Paragraph identifiers	A genuine Word numbered list: number at 0.44", text at 1.04", followed by a tab.
Claim numbering	Literal text, starting at 1, continuous, one paragraph per claim.
Abstract closing line	The figure reference written exactly as Fig.1. — no space after Fig.

6.2 Two-stage rendering

The renderer builds a typed sequence of blocks first, each carrying a kind that determines its formatting, and only then maps those blocks onto Word paragraphs. The separation has two payoffs: document structure becomes assertable in tests without parsing a binary, and the web interface gets a text preview that cannot drift from the real output because it is generated from the same sequence.

```
class Kind(StrEnum):
    COVER = "cover"           # APPLICATION / FOR / UNITED STATES...
    LABEL = "label"           # TITLE: and INVENTOR:
    DOC_TITLE = "doc_title"    # the title repeated above the body
    HEADING = "heading"        # BACKGROUND, SUMMARY, CLAIMS, ...
    NUMBERED = "numbered"      # a body paragraph carrying a list number
    CLAIM = "claim"
    FIGURE = "figure"          # the closing Fig.N. line
```

6.3 Four-digit list numbering

The bracketed paragraph identifiers are not literal text. In the source document they are Word list numbering, which matters: literal numbers would be duplicated by any list applied later, and would renumber inconsistently if a translator edited the file.

This presented a concrete obstacle. Word offers no built-in numbering format that zero-pads to four digits — decimalZero pads to two — and python-docx exposes no API for numbering definitions at all. The renderer therefore injects an ECMA-376 custom format directly into the document's numbering part.

```
<w:abstractNum w:abstractNumId="900">
  <w:multilevelType w:val="singleLevel"/>
  <w:lvl w:ilvl="0">
    <w:start w:val="1"/>
    <w:numFmt w:val="custom" w:format="0001, 0002, 0003, ..."/>
    <w:lvlText w:val="[%1]"/>
    <w:lvlJc w:val="left"/>
    <w:pPr><w:ind w:left="1498" w:hanging="864"/></w:pPr>
  </w:lvl>
</w:abstractNum>
```

The indent values are the measured geometry converted to twips: text at 1.04 inches, hanging by the 0.6 inches that separates it from the number position. One ordering constraint applies — within the numbering part, every abstract definition must precede every concrete one, so the element is inserted before the first existing definition rather than appended.

6.4 JSON output

The JSON file carries the same content in a form built for parsing rather than reading. It mirrors the domain model, not the document: no bracketed identifiers, no uppercase headings, no template ordering — those belong to the Word output and would be noise here.

The envelope carries a schema version so consumers can detect format changes rather than infer them, and the source name as provenance for human readers. Tuples are converted to lists at the boundary so that a serialize-and-parse round trip compares equal. Non-ASCII characters are preserved rather than escaped, because patent text routinely carries accented names and the file is read by people as well as machines.

7. Interfaces

Both interfaces are deliberately thin. Either could be deleted without affecting the other, because neither contains retrieval or formatting logic — a property that is easy to claim and easy to verify by looking for imports.

7.1 Command-line interface

The CLI is the primary interface. One command retrieves a patent and writes both files.

```
(.venv) Abdulrahmans-MacBook-Air-2:patent-retriever zuneccs$ clear && patent-retriever fetch US20250097171A1 --docket ABC-1234567 --client-ref REF-000123 -v
INFO patent_retriever.services.retrieval: Trying google_patents for US20250097171A1
INFO https: HTTP Request: GET https://patents.google.com/patent/US20250097171A1/en "HTTP/1.1 200 OK"
INFO patent_retriever.services.retrieval: google_patents returned a complete document for US20250097171A1
Retrieved US20250097171A1 from google_patents
output/US20250097171A1.docx
output/US20250097171A1.json
```

Figure 4. Retrieving a United States publication through Google Patents. Progress goes to standard error; only the result goes to standard output.

```
(.venv) Abdulrahmans-MacBook-Air-2:patent-retriever zuneccs$ clear && PATENT_SOURCE_ORDER=epo_ops,google_patents patent-retriever fetch US20250097171A1 -v
INFO patent_retriever.services.retrieval: Trying epo_ops for US20250097171A1
INFO patent_retriever.sources.epo_ops: epo_ops: requesting US20250097171A1 as US20250097171
INFO https: HTTP Request: POST https://ops.epo.org/3.2/auth/accessToken "HTTP/1.1 200 OK"
INFO https: HTTP Request: GET https://ops.epo.org/3.2/rest-services/published-data/publication/epodoc/US20250097171/biblio "HTTP/1.1 200 OK"
INFO https: HTTP Request: GET https://ops.epo.org/3.2/rest-services/published-data/publication/epodoc/US20250097171/claims "HTTP/1.1 404 Not Found"
WARNING patent_retriever.services.retrieval: epo_ops failed for US20250097171A1: epo_ops: OPS does not serve claims for US20250097171A1 - full text is available for EP and WO documents only.
INFO patent_retriever.services.retrieval: Trying google_patents for US20250097171A1
INFO https: HTTP Request: GET https://patents.google.com/patent/US20250097171A1/en "HTTP/1.1 200 OK"
INFO patent_retriever.services.retrieval: google_patents returned a complete document for US20250097171A1
Retrieved US20250097171A1 from google_patents
output/US20250097171A1.docx
output/US20250097171A1.json
```

Figure 5. The fallback chain. EPO OPS normalizes the reference, succeeds on bibliographic data, and is refused the claims; the warning names the reason and Google Patents completes the retrieval.

```
(.venv) Abdulrahmans-MacBook-Air-2:patent-retriever zuneccs$ clear && PATENT_SOURCE_ORDER=epo_ops patent-retriever fetch EP00000001A1 --docket ABC-1234567 --client-ref REF-000123 -v
INFO patent_retriever.services.retrieval: Trying epo_ops for EP00000001A1
INFO patent_retriever.sources.epo_ops: epo_ops: requesting EP00000001A1 as EP00000001
INFO https: HTTP Request: POST https://ops.epo.org/3.2/auth/accessToken "HTTP/1.1 200 OK"
INFO https: HTTP Request: GET https://ops.epo.org/3.2/rest-services/published-data/publication/epodoc/EP00000001/biblio "HTTP/1.1 200 OK"
INFO https: HTTP Request: GET https://ops.epo.org/3.2/rest-services/published-data/publication/epodoc/EP00000001/claims "HTTP/1.1 200 OK"
INFO https: HTTP Request: GET https://ops.epo.org/3.2/rest-services/published-data/publication/epodoc/EP00000001/description "HTTP/1.1 200 OK"
INFO patent_retriever.services.retrieval: epo_ops returned a complete document for EP00000001A1
Retrieved EP00000001A1 from epo_ops
output/EP00000001A1.docx
output/EP00000001A1.json
```

Figure 6. A European publication retrieved from EPO OPS. One authentication request serves all three endpoint calls.

```
(.venv) Abdulrahmans-MacBook-Air-2:patent-retriever zuneccs$ clear && patent-retriever sources
Configured order: google_patents
Available: google_patents, epo_ops
```

Figure 7. Configured order against available sources. The order is environment configuration; the availability list comes from the registry.

```
(.venv) Abdulrahmans-MacBook-Air-2:patent-retriever zuneccs$ clear && patent-retriever fetch NOT-A-PATENT ; echo "exit code: $?"
'NOT-A-PATENT' is not a recognizable patent number. Expected a form like 'US20250097171A1'.
exit code: 2
```

Figure 8. An unusable publication number is rejected before any network request, with exit code 2 distinguishing user error from retrieval failure.

7.2 Web interface

The web interface serves the same pipeline through a browser, for users who would not open a terminal. It adds one capability the CLI does not have: a preview of the generated document before downloading it.

PATENT RETRIEVER

Enter a patent number.

PUBLICATION NUMBER

US20250097171A1

DOCKET optional

REFERENCE optional

RETRIEVE

Figure 9. The retrieval form. Docket and client reference are optional because no patent source supplies them — they are properties of the rendering job.

PATENT RETRIEVER

Llm fine-tuning for chatbot

PUBLICATION	SOURCE	CLAIMS	INVENTORS
US20250097171A1	google_patents	20	6

[DOWNLOAD .DOCK](#) [DOWNLOAD .JSON](#)

APPLICATION FOR UNITED STATES LETTERS PATENT

TITLE: Llm fine-tuning for chatbot

INVENTOR: Yazhe Hu, Mengqing Guo, Zheng Wang, Tao Sheng, Jun Qian, Vinod Mantani

Llm fine-tuning for chatbot

BACKGROUND

[1] This disclosure claims the benefit of U.S. Provisional Patent Application Ser. No. "63/538,663" filed Sep. 15, 2023, titled "Large Language Model Fine-Tuning", inventors: Yazhe Hu, Zheng Wang, Mengqing Guo, Tao Sheng, Jun Qian, & Vinod Mantani, and assigned to the present assignee, which is incorporated by reference herein in its entirety.

[2] A large language model (LLM) is an artificial intelligence system that has been trained on vast amounts of text data to generate appropriate text responses to human language prompts. A LLM is capable of

Figure 10. A retrieved patent. The preview renders from the same block sequence as the Word file, so it cannot drift from the real output.

Retrieved documents are held in a bounded least-recently-used cache so that previewing and then downloading does not fetch the patent twice. The cache is in-process, which is adequate for a single-worker internal tool and is the one component that would need replacing in a multi-worker deployment.

8. The Generated Document

The figures in this section show generated output with Word's formatting marks enabled, so that tabs, paragraph marks, and the blank paragraphs the template requires are visible rather than inferred.

The cover page carries the only content whose vertical position is fixed by counted blank paragraphs rather than by spacing. Four empty paragraphs precede the application statement, three separate it from the title label, and one sits between the title and the inventor line. Reproducing them as spacing values would look identical on screen and diverge the moment anybody edited the file.

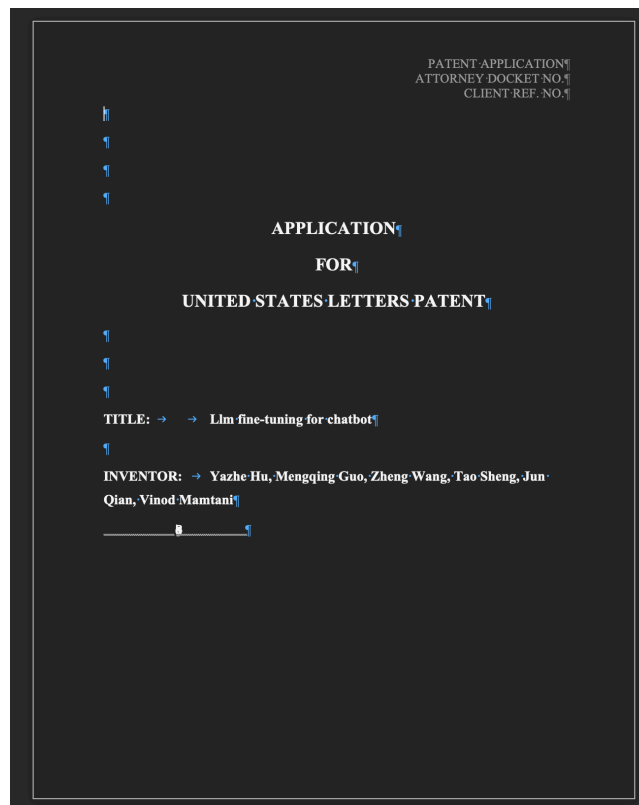


Figure 11. Cover page. The four leading paragraph marks are visible, TITLE: is followed by two tab characters, and the running header appears top-right.

In the body, each paragraph identifier is generated by Word rather than written into the text. Selecting a numbered paragraph in Word activates the numbering controls in the ribbon, which is the practical test that the injected definition is a real list and not a convincing imitation. The tab that follows each identifier is part of the list definition, not a character in the paragraph.

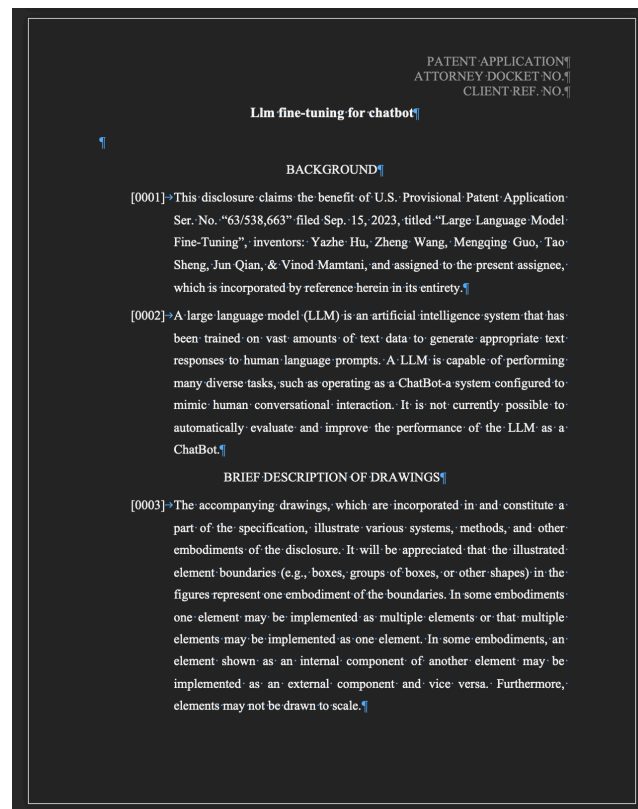


Figure 12. Body pages. Paragraph identifiers are list numbering in the required four-digit bracketed form, each followed by a tab, beneath the centred section headings the template specifies.

Output from the two sources differs in exactly one visible respect, and it is the sectioning difference described in Section 5.3. Google Patents supplies headings, so its documents split across BACKGROUND, BRIEF DESCRIPTION OF DRAWINGS, and DETAILED DESCRIPTION. EPO OPS supplies none, so its documents carry a single section that the source labels itself.

Everything else is identical, which is the point of putting a normalization step between retrieval and rendering. The renderer never learns which source produced the document it is given, and the two files differ only where the underlying data genuinely differs.

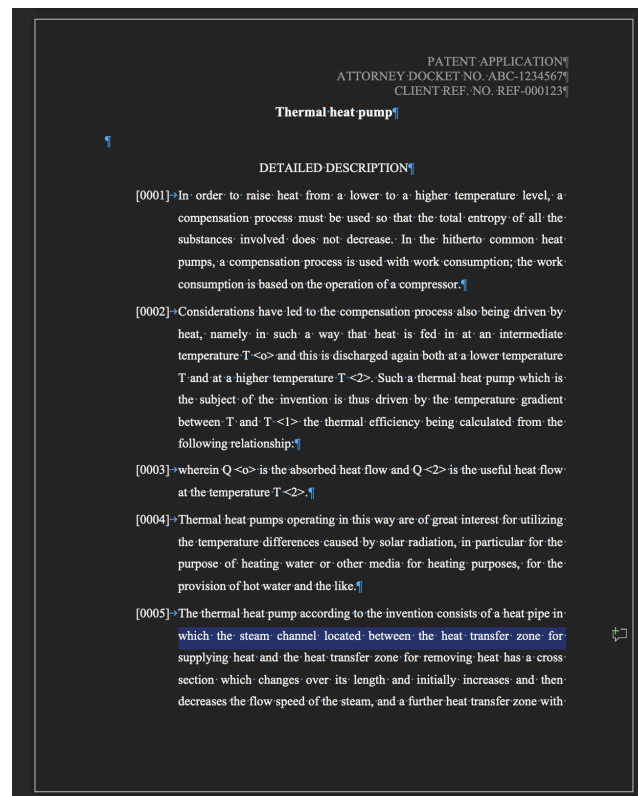


Figure 13. A European publication retrieved from EPO OPS. The description arrives without headings and is labelled as the detailed description by the source itself.

8.1 A data defect worth documenting

One test document exposes a limitation that belongs to the source rather than to this application. The English claims block that OPS returns for EP0000001 — a document published in 1978 — is a corrupt optical-character-recognition dump: several hundred characters of front-page bibliographic data rather than claims. The German block for the same document is intact.

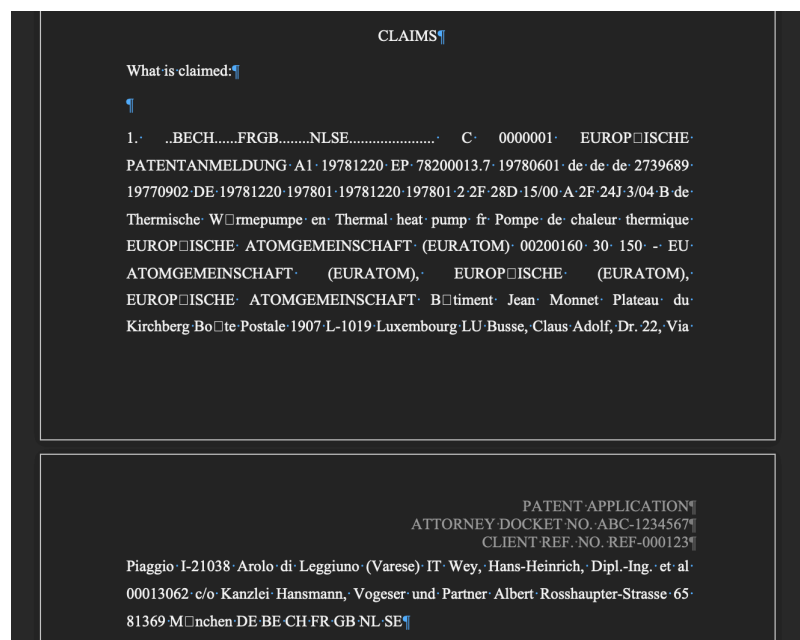


Figure 14. Corrupt source data reaching the generated document. The claim text is front-page bibliographic data supplied by the API, not a parsing error.

No amount of parsing fixes this, and detecting generically that a block of text is not really a set of claims would be guesswork. It is recorded here because a technical document that shows only its successes is not describing the system honestly.

9. Design Decisions

Each decision below had a defensible alternative. The reasoning matters more than the conclusion.

9.1 A frozen dataclass rather than Pydantic

Pydantic exists to parse and validate untrusted external data — which here is HTML and JSON, and therefore lives in the sources package, not the core. Making the domain model a Pydantic model would place a third-party import at the centre of the application and reverse the dependency direction the architecture depends on.

The counter-argument is real: Pydantic would give free JSON serialization and richer validation for less code. It was rejected because the validation required is five lines, dataclasses.asdict covers the serialization, and the dependency rule is worth more than the saving. Pydantic would be the right choice with many models, deeply nested external schemas, or a framework that already speaks it.

9.2 Completeness is a service concern

A source could decide for itself whether what it found is good enough. That would scatter one policy across every adapter and guarantee that the adapters disagree. Instead a source reports what it found or raises, and one place decides whether to keep looking.

9.3 Presentation lives in the renderer

A `to_docx` method on `PatentDocument` would be convenient and would put `python-docx` in the domain. Formatting is a separate responsibility, so it is a separate module that takes a document and returns bytes.

9.4 Returning bytes rather than writing files

The renderers return bytes; only a thin wrapper writes to disk. This keeps filesystem concerns out of the formatting logic and is what lets the web interface stream a document straight to the browser without a temporary file.

9.5 Non-English full text is used, with a warning

When a document has no English claims or description, the source uses the available language and logs a warning at a level visible without verbose output. The alternative — treating absent English as absent content — would discard the entire description of at least one test document and produce a nearly empty result.

The application prepares input for translation, so foreign-language text is usable by definition. The real risk is silently mixing languages without saying so, and the warning is what addresses it.

9.6 The country limitation is discovered, not hard-coded

The EPO source could short-circuit on a United States number and skip the network entirely, saving one request. It does not. Hard-coding a coverage rule would go stale silently if OPS extended its coverage; the cost of learning it from the API is a single wasted request on a source that is not first in the chain for those patents anyway.

9.7 Claim boundaries by prefix, not by structure

OPS nests every line of every claim under a single element, so the response structure carries no claim boundaries at all. The number prefix is the only signal available. The rule requires a space after the period so that a decimal value opening a continuation line cannot be read as a claim number — an edge case covered by an explicit test.

9.8 Selective markup stripping

Image placeholders are removed from EPO description text; other angle-bracket sequences are not. In text-only full text, sequences such as T <2> are literal subscript notation, and a general tag stripper would delete them silently and corrupt the meaning of the sentence.

10. Error Handling

Errors are typed, and the type is what drives behaviour. Without a hierarchy the service would be reduced to inspecting error message strings to decide whether to continue — fragile, and untestable.

```

PatentRetrieverError
├── InvalidPatentNumberError    -> stop immediately; the input cannot be a patent
├── InvalidPatentDocumentError -> a document was constructed in an invalid state
├── IncompleteDocumentError    -> retrieved, but missing content required to render
├── RenderError                -> the document could not be rendered
├── SourceError                -> advance the fallback chain
│   ├── PatentNotFoundError    -> this source has no record of it
│   ├── SourceUnavailableError -> network, timeout, 5xx, quota, auth
│   └── ParseError             -> the response could not be interpreted

```

Everything beneath `SourceError` means "try the next source". Everything outside it means something else has gone wrong and retrying cannot help. That single distinction is the whole fallback logic.

10.1 Mapping external failures

Condition	Raised	Effect
Authentication rejected or unreachable	<code>SourceUnavailableError</code>	Advance the chain
HTTP 404 with a country restriction	<code>PatentNotFoundError</code>	Advance the chain
HTTP 404, any other reason	<code>PatentNotFoundError</code>	Advance the chain
HTTP 403 or 429 (quota, access)	<code>SourceUnavailableError</code>	Advance the chain
HTTP 5xx	<code>SourceUnavailableError</code>	Advance the chain
Network error or timeout	<code>SourceUnavailableError</code>	Advance the chain
Response body cannot be parsed	<code>ParseError</code>	Advance the chain, logged loudly
Required content absent from a valid response	<code>ParseError</code>	Advance the chain
Source configured without credentials	<code>ValueError</code>	Configuration fault; reported to the user
Publication number malformed	<code>InvalidPatentNumberError</code>	Stop before any request

A parse error is deliberately distinct from a not-found error even though both advance the chain. A parse error usually means a source changed its structure — an event that should be visible in the logs rather than indistinguishable from an absent patent.

10.2 Exit codes

The command-line interface maps failure categories onto distinct exit codes, so that a script wrapping the tool can respond appropriately: 0 for success, 2 for a malformed publication number, 3 when no source could supply the patent, and 4 for a configuration fault such as an unknown source name or missing credentials.

11. Testing Strategy

The suite contains 203 tests and none of them touches the network. That is a design constraint rather than an aspiration: network tests are slow, flaky, and — worst of all — they fail without telling you whether your code broke or somebody else's website changed.

11.1 Principles

- Parsing is tested as a pure function of saved fixtures. Source pages and API responses are captured once by a committed script, so the fixtures can be regenerated rather than hand-edited.
- The HTTP layer is tested only for what it owns: error mapping and token caching, through a request interceptor.
- Domain logic is tested exhaustively, because it has no I/O and therefore no excuse.
- Document structure is asserted against the block sequence, not by parsing generated binaries. Only container-level properties — the numbering definition, the running header, page setup — open the file.
- Interfaces are tested with the service replaced, since their responsibility is argument handling, status codes, and file writing.

11.2 Coverage by area

Area	Focus
Domain model	Validation, immutability, ordering rules, renumbering, canonical section headings.
Patent numbers	Equivalent input forms, other jurisdictions, malformed input, country extraction.
Section mapping	Fifteen real-world heading variants, rule precedence, unrecognized headings, continuous numbering.
Google Patents	Parsing against a saved page; number, title, inventors, abstract, claims, section population, and every error mapping.
EPO OPS	Reference normalization, language selection, object-versus-list handling, claim splitting, inventor selection, text cleanup, token caching, and every error mapping.
Retrieval service	Source ordering, fallback on failure and on incompleteness, best-candidate selection, aggregated failure reporting.
Word renderer	Cover structure, tab characters, numbering continuity, section order and omission, page setup, the numbering definition, header content.
JSON renderer	Envelope fields, renumbering, round-trip equality, absence of document formatting, non-ASCII preservation.
Configuration and registry	Defaults, parsing, malformed values, unknown source names, missing credentials.
Interfaces	Exit codes, output selection, warning display, status codes, downloads, cache eviction.

11.3 Deliberately not tested

Live external services, because that would make the suite depend on credentials and a third party. Third-party library behaviour, as opposed to this application's mapping of it. Generated dataclass methods.

Section mapping is not retested inside the source tests — the sources call it rather than reimplementing it, so retesting there would exercise the wrong module.

Naming what is not covered is as much a part of a test strategy as naming what is. A reader can then judge the gaps rather than assume there are none.

12. Results and Validation

12.1 Automated checks

```
(.venv) Abdulrahmans-MacBook-Air-2:patent-retriever zunecs$ clear && python -m pytest -q
..... [ 89%]
203 passed in 1.42s [100%]
```

Figure 15. The full suite: 203 tests passing in under two seconds, with no network access.

```
(.venv) Abdulrahmans-MacBook-Air-2:patent-retriever zunecs$ clear && mypy && ruff check . && ruff format --check .
All checks passed!
41 files already formatted
```

Figure 16. Static analysis. mypy runs in strict mode across the source tree; ruff reports no lint findings and no formatting drift.

12.2 End-to-end verification

Both retrieval paths were exercised against live services. A United States publication with EPO first in the chain fails there with the country restriction and is completed by Google Patents; a European publication is retrieved from EPO OPS as a complete document with no warnings. Those runs appear as Figures 5 and 6.

12.3 Document validity

Generated documents were verified three independent ways rather than by eye. First, archive integrity, with every XML part parsed. Second, python-docx, confirming the injected four-digit numbering definition is present and the running header carries the expected values. Third, an independent Word reader on the operating system, confirming the file opens outside the library that produced it.

Document	Source	Paragraphs	Sections emitted
US20250097171A1	Google Patents	191	BACKGROUND, BRIEF DESCRIPTION OF DRAWINGS, DETAILED DESCRIPTION, CLAIMS, ABSTRACT
EP0000001A1	EPO OPS	58	DETAILED DESCRIPTION, CLAIMS, ABSTRACT
EP3000001A1	EPO OPS	206	DETAILED DESCRIPTION, CLAIMS

The third document is correctly reported as missing its abstract: OPS holds no abstract for that publication. A tool that silently invented one would be worse than a tool that says what it does not have.

12.4 Continuous integration

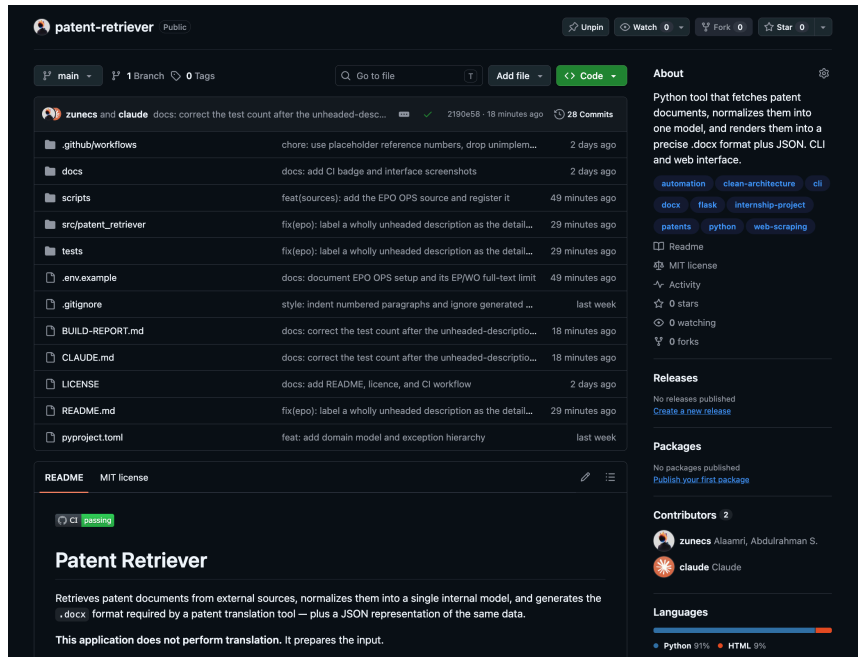


Figure 17. The public repository. The badge reflects the most recent workflow run on the default branch.

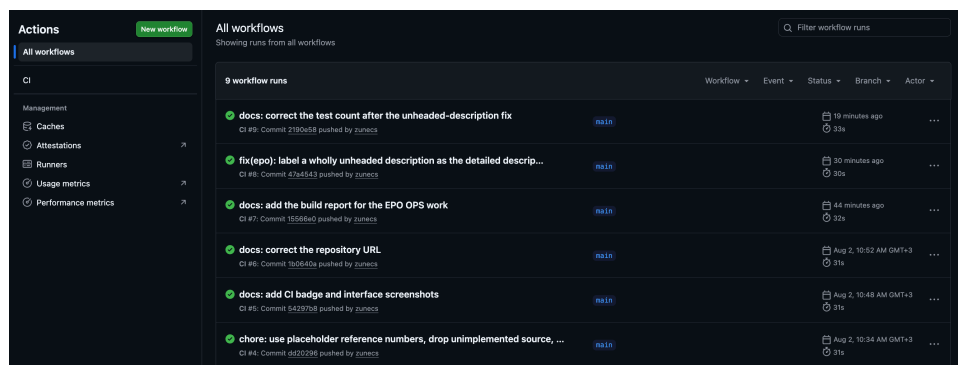


Figure 18. Continuous integration history. Every push runs linting, formatting, strict type checking, and the full suite on a clean checkout.

Continuous integration on a clean checkout is not ceremony here. It installs the package from scratch in a fresh environment, which is precisely the condition under which packaging and import-path errors surface — the class of problem that is invisible on a development machine where everything happens to be on the path already.

13. Deployment and Setup

13.1 Prerequisites

- Python 3.11 or later.
- An EPO OPS account, if European or WO coverage is required. Registration is free; a consumer key and secret are issued per registered application.
- No account, key, or approval is needed to use Google Patents.

13.2 Installation

```
git clone https://github.com/zunecs/patent-retriever.git
cd patent-retriever
python -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\activate
cp .env.example .env

pip install .                      # to use the tool
pip install -e ".[dev]"            # to work on it: tests, linting, typing
```

13.3 Configuration

Variable	Default	Purpose
PATENT_SOURCE_ORDER	google_patents	Comma-separated source priority; the first success wins.
HTTP_TIMEOUT_SECONDS	20	Per-request timeout.
OUTPUT_DIR	./output	Where generated files are written.
EPO OPS_KEY	unset	Consumer key from the EPO developer portal.
EPO OPS_SECRET	unset	Consumer secret from the EPO developer portal.

Credentials live in an ignored .env file that is never committed; a committed template documents the variables without their values. Real environment variables take precedence, so a deployment can supply secrets without a file.

13.4 Usage

```
patent-retriever fetch US20250097171A1
patent-retriever fetch US20250097171A1 --docket ABC-1234567 --client-ref REF-000123
patent-retriever fetch US20250097171A1 --json-only
patent-retriever fetch US20250097171A1 -v          # show which sources were tried
patent-retriever sources                          # configured and available sources

python -m patent_retriever.interfaces.web.app      # web interface on port 5000
```

13.5 Development

```
python -m pytest -q          # 203 tests, no network access
mypy                        # strict, source tree only
ruff check .                 # linting
ruff format .                # formatting
```

14. Limitations

Every limitation below is known and none is hidden by the implementation.

14.1 Sources

- Google Patents is scraped rather than queried. It publishes no API, its markup can change without notice, and scraping sits uneasily with its terms of use. Source order is configurable specifically so that another source can be promoted without code changes.
- EPO OPS serves no United States full text. Bibliographic data and abstracts are available; claims and description are not. US patents therefore depend on Google Patents.
- EPO descriptions arrive as a single undivided section, because the API's text-only full text contains no headings. This is accurate and matches the template, but coarser than the four-way split Google Patents supports.
- English is not guaranteed. Some documents have no English claims or description; the available language is substituted and a warning is logged.
- Source data can be corrupt. At least one document returns optical-character-recognition noise in place of its English claims, and the application faithfully reproduces it.
- No retry or backoff. A transient server error fails the source and advances the chain rather than being retried.

14.2 Output

- The template is United States specific. European and WO patents render into a document headed UNITED STATES LETTERS PATENT.
- The abstract's closing figure reference is only available from Google Patents; EPO abstracts do not carry one, so it is omitted rather than invented.
- Drawings are not retrieved. The application handles text only.

14.3 Deployment

- The web interface caches results in process memory and rebuilds its sources per request, so it re-authenticates with EPO on each retrieval. Both are adequate for a single-worker internal tool and would need attention in a multi-worker deployment.
- Quota behaviour is untested under load. Rate-limit responses are mapped correctly from simulated responses, not from a real throttling event.

15. Future Work

- A United States full-text source to complement Google Patents, giving the fallback chain a genuine second option for the jurisdiction that matters most to this use case.
- A configurable output template, so that non-US patents render with correct wording and the format can be revised without code changes.
- Retry with exponential backoff for transient failures, distinguishing them from permanent ones.
- A shared cache and a single long-lived source instance for multi-worker deployment.

- Batch retrieval, so that a list of publication numbers can be processed in one run.
- Drawing extraction, if the translation pipeline ever requires figures alongside text.

Closing note

This document describes an application built from an empty directory to a published, tested, and continuously integrated tool. Its architecture claim — that adding a data source touches two files and nothing else — was not asserted in advance and then defended; it was tested by adding a second source after the first was complete, and it held.

The parts that proved hardest were not the ones anticipated at the outset. Reproducing four-digit Word list numbering required writing document XML by hand. Determining why an API returned a misleading 404 required probing four reference formats. Both are recorded here because the reasoning is the part worth keeping.